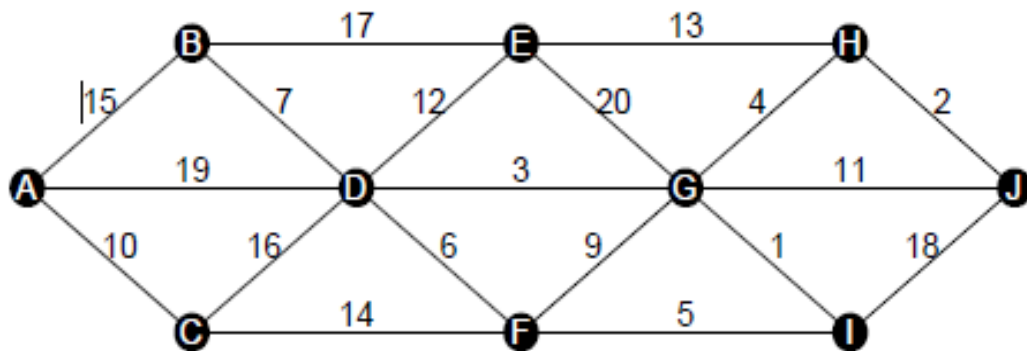


Batch – T5**Assignment No. – 7****Title – Greedy Algorithms****Student Name – Sharaneshwar Bharat Punjal****Student PRN – 23520011**

Implement Kruskal's & Prim's algorithm to find Minimum Spanning Tree (MST) of the given an undirected, connected and weighted graph.

**Kruskal's Algorithm:**

1. Sort the Edges: Sort all edges in non-decreasing order based on their weights.
2. Initialize Union-Find Structures: Create a parent array where parent[i] points to the parent of node i. Initially, every node is its own parent. Create a rank array to keep track of the tree's height to optimize the union operation.
3. Process Edges: Iterate over the sorted edges. For each edge, use the findParent function to determine the root of the source and destination vertices. If the roots are different (i.e., they are not connected), add the edge to the result list (MST). Use the unionSets function to connect the two trees.
4. Stop Condition: Stop when the number of edges in the MST is equal to $V - 1$ (where V is the number of vertices).
5. Output the Result: Print the edges included in the MST and their weights.

Code:

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

class Edge {
    char src, dest;
    int weight;

    Edge(char src, char dest, int weight) {
        this.src = src;
        this.dest = dest;
        this.weight = weight;
    }
}
```

```
public class KruskalMST {
    static int findParent(int[] parent, int i) {
        if (parent[i] == i)
            return i;
        return parent[i] = findParent(parent, parent[i]);
    }

    static void unionSets(int[] parent, int[] rank, int x, int y) {
        int xroot = findParent(parent, x);
        int yroot = findParent(parent, y);

        if (rank[xroot] < rank[yroot])
            parent[xroot] = yroot;
        else if (rank[xroot] > rank[yroot])
            parent[yroot] = xroot;
        else {
            parent[yroot] = xroot;
            rank[xroot]++;
        }
    }

    static void kruskalMST(List<Edge> edges, int V) {
        List<Edge> result = new ArrayList<>();
        int[] parent = new int[V];
        int[] rank = new int[V];

        for (int v = 0; v < V; ++v) {
            parent[v] = v;
            rank[v] = 0;
        }

        Collections.sort(edges, Comparator.comparingInt(edge -> edge.weight));

        for (Edge edge : edges) {
            int x = findParent(parent, edge.src - 'A');
            int y = findParent(parent, edge.dest - 'A');

            if (x != y) {
                result.add(edge);
                unionSets(parent, rank, x, y);
            }

            if (result.size() == V - 1) break;
        }

        System.out.println("Kruskal's MST:");
        for (Edge edge : result) {
            System.out.println(edge.src + " -- " + edge.dest + " == " + edge.weight);
        }
    }
}
```

```
}

public static void main(String[] args) {
    List<Edge> edges = new ArrayList<>();
    edges.add(new Edge('A', 'B', 15));
    edges.add(new Edge('A', 'C', 10));
    edges.add(new Edge('B', 'D', 7));
    edges.add(new Edge('B', 'E', 17));
    edges.add(new Edge('C', 'D', 19));
    edges.add(new Edge('C', 'F', 16));
    edges.add(new Edge('D', 'E', 12));
    edges.add(new Edge('D', 'F', 6));
    edges.add(new Edge('E', 'G', 20));
    edges.add(new Edge('F', 'G', 9));
    edges.add(new Edge('G', 'H', 4));
    edges.add(new Edge('G', 'I', 1));
    edges.add(new Edge('G', 'J', 11));
    edges.add(new Edge('H', 'J', 2));
    edges.add(new Edge('I', 'J', 18));
    edges.add(new Edge('F', 'I', 5));

    int V = 10;

    kruskalMST(edges, V);
}
```

Output:

Kruskal's MST:

```
G -- I == 1
H -- J == 2
G -- H == 4
F -- I == 5
D -- F == 6
B -- D == 7
A -- C == 10
D -- E == 12
A -- B == 15
```

Time Complexity – $O(V+E)$

Space Complexity – $O(V+E)$

Prim's Algorithm Code:

1. Initialize Structures: Create a key array to store the minimum weight edge for each vertex, initialized to infinity (or a large value). Create a parent array to keep track of the selected edges for the MST. Create a boolean array to track which vertices are included in the MST.

2. Set the Starting Vertex: Set the key value of the starting vertex to 0, indicating that it will be included in the MST first. Push the starting vertex into a priority queue (or min-heap) based on its key value.
3. Process Vertices: While the priority queue is not empty, extract the vertex with the minimum key value. Mark this vertex as included in the MST.
4. Update Key Values: For each adjacent vertex of the extracted vertex, if the vertex is not already included in the MST and the weight of the edge connecting them is less than the current key value of the adjacent vertex, update the key value. Set the parent of the adjacent vertex to the extracted vertex. Push the updated adjacent vertex into the priority queue.
5. Repeat Until All Vertices Are Processed: Continue processing until all vertices have been included in the MST.
6. Output the Result: Print the edges included in the MST along with their weights by traversing the parent array.

Code:

```
import java.util.*;

class Edge2 {
    int weight;
    char dest;

    Edge2(int weight, char dest) {
        this.weight = weight;
        this.dest = dest;
    }
}

public class PrimMST {

    public static void primMST(List<List<Edge2>> graph, int V, char start) {
        int[] key = new int[V];
        char[] parent = new char[V];
        boolean[] inMST = new boolean[V];

        Arrays.fill(key, Integer.MAX_VALUE);
        Arrays.fill(parent, '-');
        Arrays.fill(inMST, false);

        PriorityQueue<Edge2> pq = new PriorityQueue<>(Comparator.comparingInt(e ->
e.weight));

        int startIdx = start - 'A';
        pq.add(new Edge2(0, start));
        key[startIdx] = 0;

        while (!pq.isEmpty()) {
            char u = pq.poll().dest;
            int uIdx = u - 'A';
```

```
        inMST[uIdx] = true;

        for (Edge2 edge2 : graph.get(uIdx)) {
            char v = edge2.dest;
            int weight = edge2.weight;
            int vIdx = v - 'A';

            if (!inMST[vIdx] && key[vIdx] > weight) {
                key[vIdx] = weight;
                pq.add(new Edge2(key[vIdx], v));
                parent[vIdx] = u;
            }
        }
    }

    System.out.println("Prim's MST:");
    for (int i = 1; i < V; ++i) {
        System.out.println(parent[i] + " -- " + (char) ('A' + i) + " == " +
key[i]);
    }
}

public static void main(String[] args) {
    int V = 10;

    List<List<Edge2>> graph = new ArrayList<>(V);
    for (int i = 0; i < V; i++) {
        graph.add(new ArrayList<>());
    }

    graph.get(0).add(new Edge2(15, 'B'));
    graph.get(0).add(new Edge2(10, 'C'));
    graph.get(1).add(new Edge2(15, 'A'));
    graph.get(1).add(new Edge2(7, 'D'));
    graph.get(1).add(new Edge2(17, 'E'));
    graph.get(2).add(new Edge2(10, 'A'));
    graph.get(2).add(new Edge2(19, 'D'));
    graph.get(2).add(new Edge2(16, 'F'));
    graph.get(3).add(new Edge2(7, 'B'));
    graph.get(3).add(new Edge2(19, 'C'));
    graph.get(3).add(new Edge2(12, 'E'));
    graph.get(3).add(new Edge2(6, 'F'));
    graph.get(4).add(new Edge2(17, 'B'));
    graph.get(4).add(new Edge2(12, 'D'));
    graph.get(4).add(new Edge2(20, 'G'));
    graph.get(5).add(new Edge2(16, 'C'));
    graph.get(5).add(new Edge2(6, 'D'));
    graph.get(5).add(new Edge2(9, 'G'));
    graph.get(5).add(new Edge2(5, 'I'));
    graph.get(6).add(new Edge2(20, 'E'));
```

```
graph.get(6).add(new Edge2(9, 'F'));
graph.get(6).add(new Edge2(4, 'H'));
graph.get(6).add(new Edge2(1, 'I'));
graph.get(6).add(new Edge2(11, 'J'));
graph.get(7).add(new Edge2(4, 'G'));
graph.get(7).add(new Edge2(2, 'J'));
graph.get(8).add(new Edge2(1, 'G'));
graph.get(8).add(new Edge2(5, 'F'));
graph.get(8).add(new Edge2(18, 'J'));
graph.get(9).add(new Edge2(11, 'G'));
graph.get(9).add(new Edge2(2, 'H'));
graph.get(9).add(new Edge2(18, 'I'));
```

```
primMST(graph, V, 'A');
```

```
}
```

```
}
```

Output:

Prim's MST:

A -- B == 15

A -- C == 10

B -- D == 7

D -- E == 12

D -- F == 6

I -- G == 1

G -- H == 4

F -- I == 5

H -- J == 2

Time Complexity – $O(V+E)$

Space Complexity – $O(V+E)$

Q) How many edges does a minimum spanning tree for above example?

A minimum spanning tree (MST) for a graph with V vertices always has $V-1$ edges. In this case, the graph has 10 vertices, so the MST will have 9 edges.

Q) In a graph G . let the edge $u v$ have the least weight. is it true that $u v$ is always part of any minimum spanning tree of G ?. Justify your answers.

No, it is not always true. While an edge with the least weight might be included in an MST, it is not guaranteed unless it does not form a cycle with other edges. In some cases, other edges can provide a lower overall cost for a spanning tree if they lead to less expensive paths.

Q) Let G be a graph and T be a minimum spanning tree of G . Suppose that the weight of an edge e is decreased. How can you find the minimum spanning tree of the modified graph? What is the runtime of your solution?

If the weight of an edge e is decreased, and if e is part of the MST, then the MST remains the same. However, if e was not part of the MST, you should check whether adding this edge to the MST will replace any other edge, which can be done in $O(E)$ time by rerunning either Prim's or Kruskal's algorithm for this modified graph. The

runtime complexity of finding the MST again is $O(E \log V)$ using Kruskal's algorithm or Prim's algorithm, where E is the number of edges and V is the number of vertices.

Q) Find order of edges for Kruskal's and Prim's?

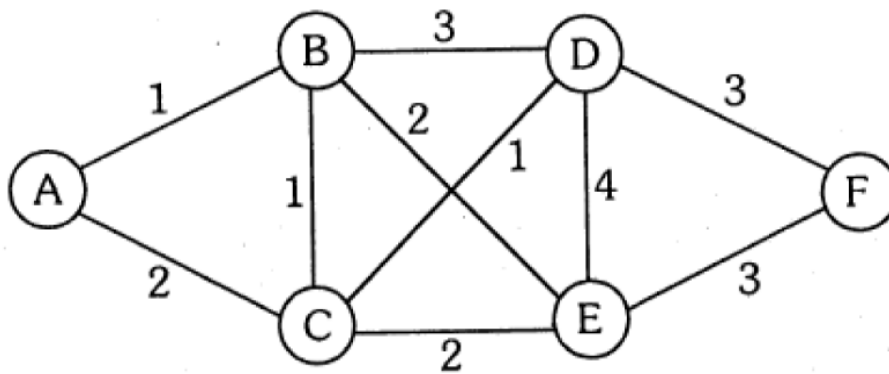
Kruskal's Algorithm Order:

(6,8), (5,6), (6,7), (4,5), (0,2), (3,5), (0,1), (7,8), (8,9), (6,8), (5,6), (6,7), (4,5), (0,2), (3,5), (0,1), (7,8), (8,9), (6,8), (5,6), (6,7), (4,5), (0,2), (3,5), (0,1), (7,8), (8,9)

Prim's Algorithm Order:

(0,2), (2,5), (5,6), (6,7), (6,8), (0,1), (1,3), (3,4), (8,9), (0,2), (2,5), (5,6), (6,7), (6,8), (0,1), (1,3), (3,4), (8,9), (0,2), (2,5), (5,6), (6,7), (6,8), (0,1), (1,3), (3,4), (8,9)

2) From a given vertex in a weighted connected graph, implement shortest path finding Dijkstra's algorithm.



Algorithm:

1. Initialize Distances: Create an array `dist` to store the shortest distance from the start vertex to each vertex. Initialize all distances as infinity (or a large number) except for the start vertex, which should be initialized to 0.
2. Use a Priority Queue: Use a priority queue (min-heap) to efficiently get the next vertex with the smallest distance. Push the start vertex with a distance of 0 into the priority queue.
3. Process the Queue: While the priority queue is not empty, repeat the following steps:
 - Extract the vertex u from the priority queue that has the smallest distance.
 - For each neighbor v of vertex u , check if the distance to v through u is smaller than the known distance to v .
 - If it is, update the distance to v in the `dist` array and push v into the priority queue with the updated distance.
4. Stop Condition: The algorithm continues until all vertices have been processed and the priority queue is empty.
5. Output the Result: After processing all vertices, print the shortest distances from the start vertex to all other vertices.

Code:

```
import java.util.*;

public class Dijkstra {
    static class Edge {
        int weight;
        int vertex;

        Edge(int weight, int vertex) {
            this.weight = weight;
            this.vertex = vertex;
        }
    }

    public static void dijkstra(List<List<Edge>> graph, int V, int start) {
        PriorityQueue<Edge> pq = new PriorityQueue<>(Comparator.comparingInt(e ->
e.weight));
        int[] dist = new int[V];
        Arrays.fill(dist, Integer.MAX_VALUE);
        pq.offer(new Edge(0, start));
        dist[start] = 0;

        while (!pq.isEmpty()) {
            int u = pq.poll().vertex;

            for (Edge edge : graph.get(u)) {
                int v = edge.vertex;
                int weight = edge.weight;

                if (dist[u] + weight < dist[v]) {
                    dist[v] = dist[u] + weight;
                    pq.offer(new Edge(dist[v], v));
                }
            }
        }

        System.out.printf("%-10s%-20s\n", "Vertex", "Distance from Start Vertex");
        for (int i = 0; i < V; i++) {
            System.out.printf("%-10c%-20d\n", (char) ('A' + i), dist[i]);
        }
    }

    public static void main(String[] args) {
        int V = 6;

        List<List<Edge>> graph = new ArrayList<>(V);
        for (int i = 0; i < V; i++) {
            graph.add(new ArrayList<>());
        }
    }
}
```

```
graph.get(0).add(new Edge(1, 1)); // A -> B (weight 1)
graph.get(0).add(new Edge(2, 2)); // A -> C (weight 2)
graph.get(1).add(new Edge(1, 0)); // B -> A (weight 1)
graph.get(1).add(new Edge(3, 3)); // B -> D (weight 3)
graph.get(1).add(new Edge(2, 2)); // B -> C (weight 2)
graph.get(2).add(new Edge(2, 0)); // C -> A (weight 2)
graph.get(2).add(new Edge(4, 4)); // C -> E (weight 4)
graph.get(2).add(new Edge(1, 1)); // C -> B (weight 1)
graph.get(3).add(new Edge(1, 4)); // D -> E (weight 1)
graph.get(3).add(new Edge(5, 5)); // D -> F (weight 5)
graph.get(4).add(new Edge(2, 2)); // E -> C (weight 2)
graph.get(4).add(new Edge(3, 3)); // E -> B (weight 3)
graph.get(4).add(new Edge(5, 5)); // E -> F (weight 5)
graph.get(5).add(new Edge(4, 3)); // F -> E (weight 4)
graph.get(5).add(new Edge(5, 4)); // F -> D (weight 5)
```

```
Scanner scanner = new Scanner(System.in);
System.out.print("Enter the starting vertex (A-F): ");
char startVertex = scanner.next().charAt(0);
dijkstra(graph, V, startVertex - 'A');
scanner.close();
```

```
}
```

```
}
```

Output:

Enter the starting vertex (A-F): A

Vertex	Distance from Start Vertex
--------	----------------------------

A	0
B	1
C	2
D	4
E	5
F	9

Time Complexity - $O((V+E)\log V)$

Space Complexity - $O(V+E)$

Q) Show that Dijkstra's algorithm doesn't work for graphs with negative weight edges.

Dijkstra's algorithm assumes that once a node's shortest distance is found, it doesn't need to be updated again. This assumption breaks down in the case of negative weight edges.

Q) Modify the Dijkstra's algorithm to find shortest path.

To modify Dijkstra's algorithm to not only find the shortest distance from the source vertex to all other vertices but also to reconstruct the shortest path, we need to maintain a parent array that keeps track of the previous vertex for each vertex in the shortest path tree. This way, once we have computed the shortest distances, we can trace back the path from the destination vertex to the source.

Step-by-Step Algorithm

1. **Initialize Distances and Parent Array:** Create an array `dist` to store the shortest distance from the source vertex to each vertex. Initialize all distances as infinity (or a large number) except for the source vertex, which should be initialized to 0. Also, create a parent array to store the previous vertex for each vertex in the shortest path.
2. **Priority Queue:** Use a priority queue (min-heap) to always expand the vertex with the smallest known distance.
3. **Relax Edges:** While the priority queue is not empty, pop the vertex with the smallest distance. For each of its adjacent vertices, check if the distance can be relaxed (i.e., if the distance to the current vertex plus the edge weight is less than the known distance to the adjacent vertex). If it can be relaxed, update the distance and set the parent of the adjacent vertex to the current vertex. Push the updated adjacent vertex into the priority queue.
4. **Reconstruct the Path:** After the algorithm completes, you can reconstruct the shortest path from the source to any vertex by following the parent pointers from the destination back to the source.
5. **Output the Result:** Print the shortest distance and the path from the source to each vertex.